

00 XCPC 板子

00 Templates

· header

```
C++
#include<bits/stdc++.h>
using namespace std;
using ll = long long; using ull = unsigned long long; #define sz(a) int((a).size())
#define pii pair<int, int>
#define pll pair<ll, ll>
#define all(a) (a).begin(), (a).end()
const int inf = (1 << 30);
const ll INF = (1ll << 60);
const double PI = acos(-1);

template<class A> string to_string(const A& v) {
    string s = "{";
    for (const auto& x : v) s += ((s).size() > 1 ? ", " : "") + to_string(x);
    return s += "}";
}

template<class A, class B> string to_string(const pair<A, B>& p) { return "(" +
to_string(p.first) + ", " + to_string(p.second) + ")"; }
void debug_out() { cerr << '\n'; }

template<class T, class... U>
void debug_out(const T& x, const U&... args) {
    cerr << ' ' << to_string(x);
    debug_out(args...);
}

// 91: red 93: yellow 96: blue
#define sc(x) cerr << "\033[" << x << "m"
#define debug(...) sc(96), cerr << "[" << #__VA_ARGS__ << "]:",
debug_out(__VA_ARGS__), sc(39)

void work(){
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T = 1;
    cin >> T;
    while(T--) work();
    return 0;
}
```

· int128_IO

```
C++
using i128 = __int128_t;
using u128 = __uint128_t;
```

```
ostream& operator<< (ostream& os, i128 x){
    if(x < 0){
        os << '-';
        x = -x;
    }
    if(x > 9) os << x / 10;
    os << int(x % 10);
    return os;
}
```

· Custom_Compare (常见结构体排序写法)

1. 重载运算符

```
struct Node{
    int x, y, id;
    bool operator < (const Node& ot) const {
        if(x != ot.x) return x < ot.x;
        if(y != ot.y) return y > ot.y;
        return id < ot.id;
    }
};

void work(){
    vector<Node> a = {
        {2, 3, 1},
        {1, 5, 2},
        {2, 1, 3}
    };
    sort(all(a));
    for(auto [x, y, id] : a){
        cout << x << ' ' << y << ' ' << id << '\n';
    }
}
```

C++

2.priority_queue: 符号相反

由于默认大根堆, operator < 里面 “小于” 的会被放下面

```
struct Node{
    int val, id;
    bool operator < (const Node& ot) const {
        if(val != ot.val) return val < ot.val; // val 大的优先
        return id > ot.id; // val 相同, id 小的优先
    }
};

priority_queue<Node> pq;
```

C++

3. set 内置排序

```
struct Cmp{
    bool operator () (const Node& a, const Node& b) const {
        if(a.x != b.x) return a.x < b.x;
```

C++

```

        if(a.y != b.y) return a.y > b.y;
        return a.id < b.id;
    }
};
set<Node, Cmp> st;

```

4. 直接用 lambda

```

auto cmp = [](const Node& a, const Node& b) → bool {
    if(a.x != b.x) return a.x < b.x;
    if(a.y != b.y) return a.y > b.y;
    return a.id < b.id;
};
set<Node, decltype(cmp)> st(cmp);

```

C++

5. 区别：

```

// sort / set: 谁小谁在前
return a.x < b.x;

// priority_queue: 谁“大”谁在堆顶
return a.x < b.x;

```

C++

01 Data_Structure

· BIT 树状数组

1. 单点加, 区间查询

适用背景:

维护一个数组 a , 支持:

```
1 x v      a[x] += v
2 l r      查询 sum[l, r]
```

C++

这是最基础的树状数组。

常见应用:

动态维护前缀和

单点修改, 区间求和

逆序对统计

扫描线辅助统计

频率数组维护

```
struct BIT{ // 单点加, 区间查询 (BIT_Point_Add____Range_Query)
    int n;
    vector<ll> tr;
    BIT(int _n = 0): n(_n), tr(_n + 2) {} // 初始化
    // void init(int _n){ // 全局 bit 初始化才需要
    //     n = _n;
    //     tr.assign(n + 2, 0);
    // }
    void add(int x, ll v){ // 单点加
        for(; x <= n; x += x & -x) tr[x] += v;
    }
    ll query(int x){ // 单点查询: 查前缀和 a[1, x]
        ll ans = 0;
        for(; x >= 1; x -= x & -x) ans += tr[x];
        return ans;
    }
    ll query(int l, int r){ // 区间查询: 即区间和 query[x, y]
        return query(r) - query(l - 1);
    }
};
// BIT bit; // 全局 bit
```

C++

```
void work(){
    int n, q;
    cin >> n >> q;
    BIT bit(n);
    for(int i = 1; i <= n; ++i){
        ll x;
        cin >> x;
```

```

        bit.add(i, x);
    }
    while(q--){
        int op;
        cin >> op;
        if(op == 1){
            int x;
            ll v;
            cin >> x >> v;
            bit.add(x, v);
        }else{
            int l, r;
            cin >> l >> r;
            cout << bit.query(l, r) << '\n';
        }
    }
}

```

2. 区间加, 单点查询

适用背景:

维护一个数组 a , 支持:

```

1 l r v    a[l, r] += v
2 x        查询 a[x]

```

核心思想是维护差分数组 d 。

C++

如果:

```

a[i] = d[1] + d[2] + ... + d[i]
那么区间加:  a[l, r] += v
等价于: d[l] += v      d[r + 1] -= v

```

C++

常见应用:

区间整体加

最后查询某些点的值

离线区间覆盖统计

差分思想 + BIT 动态化

```

struct BIT{ // 区间加, 单点查询 (BIT_Range_Add___Point_Query)
    int n;
    vector<ll> tr;
    BIT(int _n = 0): n(_n), tr(_n + 2) {} // 初始化
    void add(int x, ll v){ // 单点加
        for(; x <= n; x += x & -x) tr[x] += v;
    }
    void add(int l, int r, ll v){ // 区间加
        add(l, v);
        add(r + 1, -v);
    }
}

```

C++

```

    }
    ll query(int x){ // 单点查询: 查前缀和 a[1, x]
    ll ans = 0;
    for(; x ≥ 1; x -= x & -x) ans += tr[x];
    return ans;
    }
};

void work(){
    int n, q;
    cin >> n >> q;
    BIT bit(n);
    for(int i = 1; i ≤ n; ++i){
        ll x;
        cin >> x;
        bit.add(i, i, x);
    }

    while(q--){
        int op;
        cin >> op;
        if(op == 1){
            int l, r;
            ll v;
            cin >> l >> r >> v;
            bit.add(l, r, v);
        }else{
            int x;
            cin >> x;
            cout << bit.query(x) << '\n';
        }
    }
}
}

```

3. 区间加, 区间查询

适用背景:

维护一个数组 a , 支持:

1 l r v $a[l, r] += v$

2 l r 查询 $\text{sum}[l, r]$

这个功能比前两个都强, 用两个 BIT 维护。

核心公式:

设差分数组为 d 。

$a[1] + a[2] + \dots + a[x] = \sum d[i] (x - i + 1) = (x + 1) \sum d[i] - \sum d[i] i$

所以维护两个树状数组: $s1$ 维护 $d[i]$ $s2$ 维护 $d[i] i$

前缀和: $\text{sum}(x) = (x + 1) * s1.\text{query}(x) - s2.\text{query}(x)$

C++

```

struct BIT{ // 区间加, 区间查询 (BIT_Range_Add___Range_Query)
    int n;
    vector<ll> tr;
    BIT(int _n = 0): n(_n), tr(_n + 2) {}
    void add(int x, ll v){ // 单点加

```

```

        for(; x ≤ n; x += x & -x) tr[x] += v;
    }

    ll query(int x){ // 单点查询: 查前缀和 a[1, x]
    ll ans = 0;
    for(; x ≥ 1; x -= x & -x) ans += tr[x];
    return ans;
}

};

struct RBIT{
    int n;
    BIT s1, s2; // 用两个树状数组维护 "区间加、区间查" 功能
    RBIT(int _n): n(_n), s1(_n), s2(_n) {}
    void add(int x, ll v){
        s1.add(x, v), s2.add(x, v * x); // 两次单点加
    }

    void add(int l, int r, ll v){
        add(l, v), add(r + 1, -v); // 两次单点查询
    }

    ll query(int x){
        return (x + 1) * s1.query(x) - s2.query(x);
    }

    ll query(int l, int r){
        return query(r) - query(l - 1);
    }
};

void work(){
    int n, q;
    cin >> n >> q;
    RBIT bit(n);
    for(int i = 1; i ≤ n; ++i){
        ll x;
        cin >> x;
        bit.add(i, i, x);
    }

    while(q--){
        int op;
        cin >> op;
        if(op == 1){
            int l, r;
            ll v;
            cin >> l >> r >> v;
            bit.add(l, r, v);
        }else{
            int l, r;
            cin >> l >> r;
            cout << bit.query(l, r) << '\n';
        }
    }
}

```

4. 权值树状数组 / 求第 k 小

适用背景：

树状数组维护的不是原数组，而是“值域频率”。

例如值域是 $1 \sim n$ ，支持：

插入一个数 x
删除一个数 x
查询 $\leq x$ 的数有多少个
查询当前第 k 小

C++

常见应用：

动态第 k 小

逆序对

排名查询

离散化后维护频率

多重集合替代品

```
struct BIT_Kth{
    int n;
    vector<int> tr;
    BIT_Kth(int _n): n(_n), tr(_n + 2) {}
    void add(int x, int v){ // 单点加
        for(; x ≤ n; x += x & -x) tr[x] += v;
    }

    int query(int x){ // 单点查询：查前缀和 a[1, x]
        int ans = 0;
        for(; x ≥ 1; x -= x & -x) ans += tr[x];
        return ans;
    }

    int kth(int k){
        int x = 0;
        for(int i = 1 << __lg(n); i; i >>= 1){
            if(x + i ≤ n && tr[x + i] < k){
                x += i;
                k -= tr[x];
            }
        }
        return x + 1;
    }
};
```

C++

```
void work(){
    int n, q;
    cin >> n >> q;
    BIT_Kth bit(n);
    while(q--){
        int op, x;
        cin >> op >> x;
```

```

        if(op == 1) bit.add(x, 1);           // 插入 x
        else if(op == 2) bit.add(x, -1);     // 删除 x
        else if(op == 3) cout << bit.query(x) << '\n'; // ≤ x 的数量
        else cout << bit.kth(x) << '\n';    // 第 x 小
    }
}

```

5. 二维树状数组

适用背景:

维护一个矩阵 a, 支持:

单点加, 矩阵查询

常见应用:

二维前缀和动态版

矩阵单点修改, 矩形求和

平面点统计

离线扫描二维偏序

```

struct BIT_2D{
    int n, m;
    vector<vector<ll>> tr;
    BIT_2D(int _n, int _m): n(_n), m(_m), tr(n + 2, vector<ll>(m + 2)) {}
    void add(int x, int y, ll v){
        for(int i = x; i ≤ n; i += i & -i){
            for(int j = y; j ≤ m; j += j & -j){
                tr[i][j] += v;
            }
        }
    }
    ll query(int x, int y){
        ll ans = 0;
        for(int i = x; i ≥ 1; i -= i & -i){
            for(int j = y; j ≥ 1; j -= j & -j){
                ans += tr[i][j];
            }
        }
        return ans;
    }
    ll query(int x1, int y1, int x2, int y2){
        return query(x2, y2) - query(x1 - 1, y2) - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
    }
};

void work(){
    int n, m, q;
    cin >> n >> m >> q;
    BIT_2D bit(n, m);
    for(int i = 1; i ≤ n; ++i){
        for(int j = 1; j ≤ m; ++j){
            ll x;

```

```

        cin >> x;
        bit.add(i, j, x);
    }
}
while(q--){
    int op;
    cin >> op;
    if(op == 1){
        int x, y;
        ll v;
        cin >> x >> y >> v;
        bit.add(x, y, v);
    }else{
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        cout << bit.query(x1, y1, x2, y2) << '\n';
    }
}
}

```

适用背景:

普通 BIT 维护的是 "和", 但如果操作是单调更新, 也可以维护最大值。

6. 前缀最大值树状数组

常见应用:

LIS 优化

DP 转移最大值

权值压缩后求前缀最优

偏序 DP

注意: 这种 BIT 一般不支持普通删除, 也不支持随便改小。

```

struct BIT_Max{
    int n;
    vector<ll> tr;
    BIT_Max(int _n = 0): n(_n), tr(_n + 2, -INF) {}
    void update(int x, ll v){
        for(; x ≤ n; x += x & -x) tr[x] = max(tr[x], v);
    }
    ll query(int x){
        ll ans = -INF;
        for(; x ≥ 1; x -= x & -x) ans = max(ans, tr[x]);
        return ans;
    }
};

void work(){
    int n;
    cin >> n;
    vector<int> a(n + 1), b;
    vector<ll> w(n + 1);
}

```

C++

```

for(int i = 1; i ≤ n; ++i){
    cin >> a[i] >> w[i];
    b.push_back(a[i]);
}
sort(all(b));
b.erase(unique(all(b)), b.end());
BIT_Max bit(sz(b));
ll ans = 0;
for(int i = 1; i ≤ n; ++i){
    int x = lower_bound(all(b), a[i]) - b.begin() + 1;
    ll cur = max(0ll, bit.query(x - 1)) + w[i];
    bit.update(x, cur);
    ans = max(ans, cur);
}
cout << ans << '\n';
}

```

· DSU 并查集

精简版：

```

struct DSU{
    vector<int> te, sz; // team
    DSU(int n): te(n){iota(all(te), 0);}
    int find(int u){return u = te[u] ? u : te[u] = find(te[u]);}
};
DSU U(n);
if(U.find(u) ≠ U.find(v)) U.te[U.te[u]] = U.te[v];

```

C++

全面版（含sz, unity）：

```

struct DSU{
    vector<int> te, sz; // team
    DSU(int n): te(n), sz(n, 1){iota(all(te), 0);}
    int find(int u){return u = te[u] ? u : te[u] = find(te[u]);}
    bool unity(int u, int v){ return(u = find(u)) == (v = find(v)) ? 0 : ((te[u] = v),
sz[v] += sz[u], 1);}
};
//兼顾了动态开点、按秩（大小）合并、路径压缩、连通块大小统计、环判定

```

C++

滑动窗口

适用场景：求满足某种条件的连续子数组（如求和大于等于 S 的最短子数组，或无重复字符的最长子串）。

核心思想：维护一个左指针 left 和右指针 right，像一条毛毛虫一样交替向前爬行。

这是一个非常通用的变长滑动窗口模板：

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

int slidingWindowTemplate(vector<int>& nums, int target) {
    int n = nums.size();
    int left = 0; // 左指针
    int min_len = INT_MAX; // 记录最短长度，初始设为无穷大
    int window_sum = 0; // 窗口内的具体状态
    for (int right = 0; right < n; right++) {
        // 1. 右指针主动吃进元素，更新窗口状态
        window_sum += nums[right];
        // 2. 判断窗口是否已经【达标/合法】（比如和  $\geq$  target 了）
        // 一旦达标，我们就尝试缩小窗口，看看能不能更短！
        while (window_sum  $\geq$  target) {
            // 3. 此时窗口是【合法】的，赶紧记录当前的短度！
            // 注意：更新答案的代码在 while 里面！
            min_len = min(min_len, right - left + 1);
            // 记录完之后，尝试把左边的元素踢出去，看剩下的还达不达标
            window_sum = nums[left];
            left++;
        }
    }
    // 如果 min_len 还是无穷大，说明压根没找到合法的窗口，返回 0
    return min_len == INT_MAX ? 0 : min_len;
}
```

C++

· SegTree 线段树

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
#define sz(a) int((a).size())
#define pii pair<int, int>
#define pll pair<ll, ll>
#define all(a) (a).begin(), (a).end()
const int inf = (1 << 30);
const ll INF = (1ll << 60);
const double PI = acos(-1);

struct SegTree{
    int n;
    vector<ll> a, sum, mx, mn, add, tag;
    vector<int> cov;
```

C++

```

SegTree(int _n = 0){
    if(_n) init(_n);
}

SegTree(const vector<ll>& _a){
    init(_a);
}

void init(int _n){
    n = _n;
    a.assign(n + 1, 0);
    sum.assign((n + 1) << 2, 0);
    mx.assign((n + 1) << 2, 0);
    mn.assign((n + 1) << 2, 0);
    add.assign((n + 1) << 2, 0);
    tag.assign((n + 1) << 2, 0);
    cov.assign((n + 1) << 2, 0);
}

void init(const vector<ll>& _a){
    a = _a;
    n = sz(a) - 1;
    sum.assign((n + 1) << 2, 0);
    mx.assign((n + 1) << 2, 0);
    mn.assign((n + 1) << 2, 0);
    add.assign((n + 1) << 2, 0);
    tag.assign((n + 1) << 2, 0);
    cov.assign((n + 1) << 2, 0);
    build(1, 1, n);
}

void pull(int p){
    sum[p] = sum[p << 1] + sum[p << 1 | 1];
    mx[p] = max(mx[p << 1], mx[p << 1 | 1]);
    mn[p] = min(mn[p << 1], mn[p << 1 | 1]);
}

void apply_add(int p, int l, int r, ll v){
    sum[p] += v * (r - l + 1);
    mx[p] += v;
    mn[p] += v;
    if(cov[p]) tag[p] += v;
    else add[p] += v;
}

void apply_set(int p, int l, int r, ll v){
    sum[p] = v * (r - l + 1);
    mx[p] = mn[p] = v;
    tag[p] = v;
    cov[p] = true;
    add[p] = 0;
}

void push(int p, int l, int r){
    if(l == r) return;

```

```

    int mid = (l + r) >> 1;
    if(cov[p]){
        apply_set(p << 1, l, mid, tag[p]);
        apply_set(p << 1 | 1, mid + 1, r, tag[p]);
        cov[p] = false;
    }
    if(add[p]){
        apply_add(p << 1, l, mid, add[p]);
        apply_add(p << 1 | 1, mid + 1, r, add[p]);
        add[p] = 0;
    }
}

void build(int p, int l, int r){
    if(l == r){
        sum[p] = mx[p] = mn[p] = a[l];
        return;
    }

    int mid = (l + r) >> 1;
    build(p << 1, l, mid);
    build(p << 1 | 1, mid + 1, r);
    pull(p);
}

void range_add(int p, int l, int r, int x, int y, ll v){
    if(x ≤ l && r ≤ y){
        apply_add(p, l, r, v);
        return;
    }

    push(p, l, r);
    int mid = (l + r) >> 1;
    if(x ≤ mid) range_add(p << 1, l, mid, x, y, v);
    if(y > mid) range_add(p << 1 | 1, mid + 1, r, x, y, v);
    pull(p);
}

void range_set(int p, int l, int r, int x, int y, ll v){
    if(x ≤ l && r ≤ y){
        apply_set(p, l, r, v);
        return;
    }

    push(p, l, r);
    int mid = (l + r) >> 1;
    if(x ≤ mid) range_set(p << 1, l, mid, x, y, v);
    if(y > mid) range_set(p << 1 | 1, mid + 1, r, x, y, v);
    pull(p);
}

ll query(int p, int l, int r, int x, int y){
    if(x ≤ l && r ≤ y) return sum[p];

    push(p, l, r);

```

```

    ll ans = 0;
    int mid = (l + r) >> 1;
    if(x ≤ mid) ans += query(p << 1, l, mid, x, y);
    if(y > mid) ans += query(p << 1 | 1, mid + 1, r, x, y);
    return ans;
}

ll qmax(int p, int l, int r, int x, int y){
    if(x ≤ l && r ≤ y) return mx[p];

    push(p, l, r);
    ll ans = -INF;
    int mid = (l + r) >> 1;
    if(x ≤ mid) ans = max(ans, qmax(p << 1, l, mid, x, y));
    if(y > mid) ans = max(ans, qmax(p << 1 | 1, mid + 1, r, x, y));
    return ans;
}

ll qmin(int p, int l, int r, int x, int y){
    if(x ≤ l && r ≤ y){
        return mn[p];
    }

    push(p, l, r);
    ll ans = INF;
    int mid = (l + r) >> 1;
    if(x ≤ mid) ans = min(ans, qmin(p << 1, l, mid, x, y));
    if(y > mid) ans = min(ans, qmin(p << 1 | 1, mid + 1, r, x, y));
    return ans;
}

void add_point(int x, ll v){
    range_add(1, 1, n, x, x, v);
}

void set_point(int x, ll v){
    range_set(1, 1, n, x, x, v);
}

void add(int l, int r, ll v){
    range_add(1, 1, n, l, r, v);
}

void setv(int l, int r, ll v){
    range_set(1, 1, n, l, r, v);
}

ll query(int l, int r){
    return query(1, 1, n, l, r);
}

ll qmax(int l, int r){
    return qmax(1, 1, n, l, r);
}

ll qmin(int l, int r){

```

```

        return qmin(1, 1, n, l, r);
    }
};

void work(){
    int n, q;
    cin >> n >> q;
    vector<ll> a(n + 1);
    for(int i = 1; i ≤ n; ++i) cin >> a[i];

    SegTree seg(a); // 建树

    while(q--){
        int op;
        cin >> op;

        if(op == 1){
            int l, r;
            ll v;
            cin >> l >> r >> v;
            seg.add(l, r, v);           // 区间加
        }else if(op == 2){
            int l, r;
            ll v;
            cin >> l >> r >> v;
            seg.setv(l, r, v);         // 区间赋值
        }else if(op == 3){
            int x;
            ll v;
            cin >> x >> v;
            seg.add_point(x, v);       // 单点加
        }else if(op == 4){
            int x;
            ll v;
            cin >> x >> v;
            seg.set_point(x, v);       // 单点赋值
        }else if(op == 5){
            int l, r;
            cin >> l >> r;
            cout << seg.query(l, r) << '\n'; // 区间和
        }else if(op == 6){
            int l, r;
            cin >> l >> r;
            cout << seg.qmax(l, r) << '\n'; // 区间最大值
        }else{
            int l, r;
            cin >> l >> r;
            cout << seg.qmin(l, r) << '\n'; // 区间最小值
        }
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T = 1;

```

```
    cin >> T;
    while(T--) work();
    return 0;
}
```

02 Graph_Theory

- 00_Graph_Basic
- 01_Graph_Traversal
- 02 DAG (Topo_Sort, DAG DP)
- 03 Shortest_Path (Dijkstra, SPFA, Floyd)

